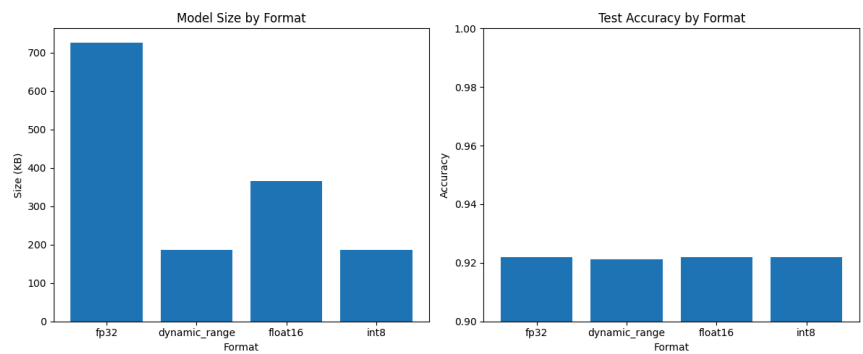


Quantization

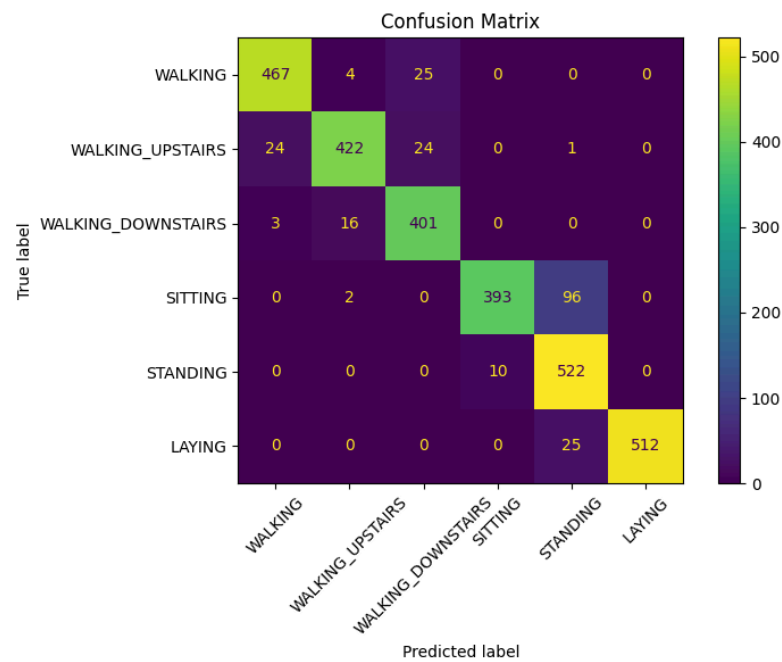
your completed notebook,

- the generated `.tflite` files,
- screenshots or output cells showing the final comparison table,

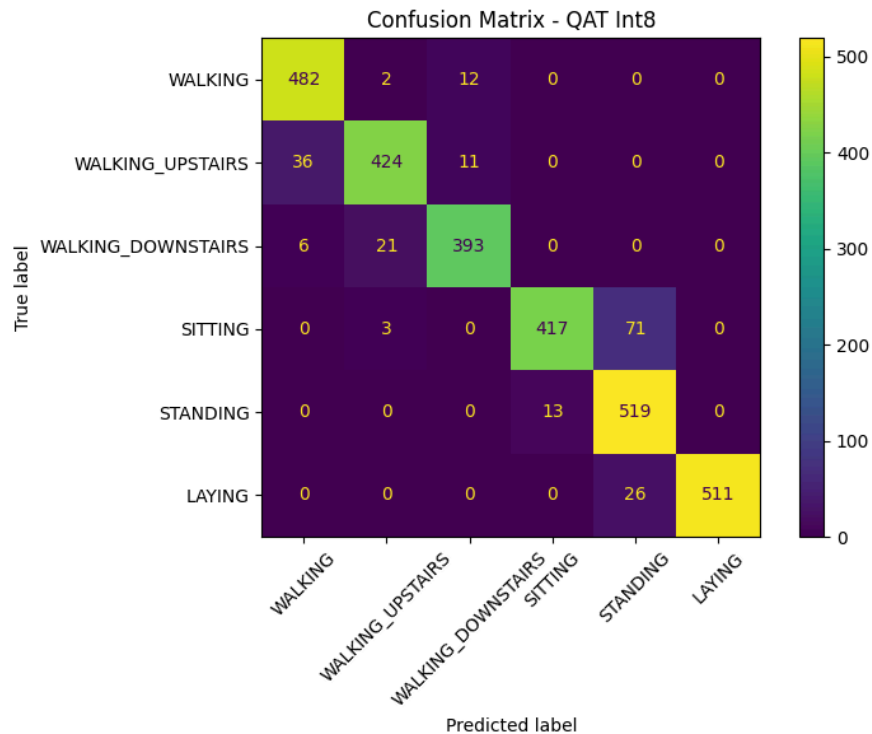


	Model Family	Format	Test Accuracy	Model Size (KB)
0	Baseline DNN	fp32	0.921955	726.660156
1	Baseline DNN	dynamic_range	0.921276	186.054688
2	Baseline DNN	float16	0.921955	365.628906
3	Baseline DNN	int8	0.921955	185.273438

- confusion matrices for the
 - baseline model



- and the final int8 model you want to highlight,



- and short written observations answering the summary questions.

Make sure your notebook runs from top to bottom without errors.

13. Summary Questions

Write short answers to the following:

1. Which quantization method gave the smallest model size?

Int8 at 185.27 KB

2. Which quantization method gave the best accuracy among the TensorFlow Lite models?

FP32, float16, and int8 are all tied at 92.1955% accuracy

3. Did QAT improve the final int8 model compared with PTQ int8?

	Model	Test Accuracy	Model Size (KB)
0	PTQ Int8	0.921955	185.273438
1	QAT Int8	0.931795	186.164062

Yes, technically. We got a 1% accuracy improvement in exchange for an extra KB of size. Whether it's "better" depends on how constrained you are for space.

4. Why is this dataset a good fit for a DNN-based TinyML workflow?

It has a small input dimensionality, data can easily be collected on low storage devices, and doesn't require much input processing at all

Compare this to my CV project classifying tiny objects that barely take up space in the camera and require color vision to be recognizable. Woof.

5. If you were deploying this model on a resource-constrained device, which version would you choose and why?

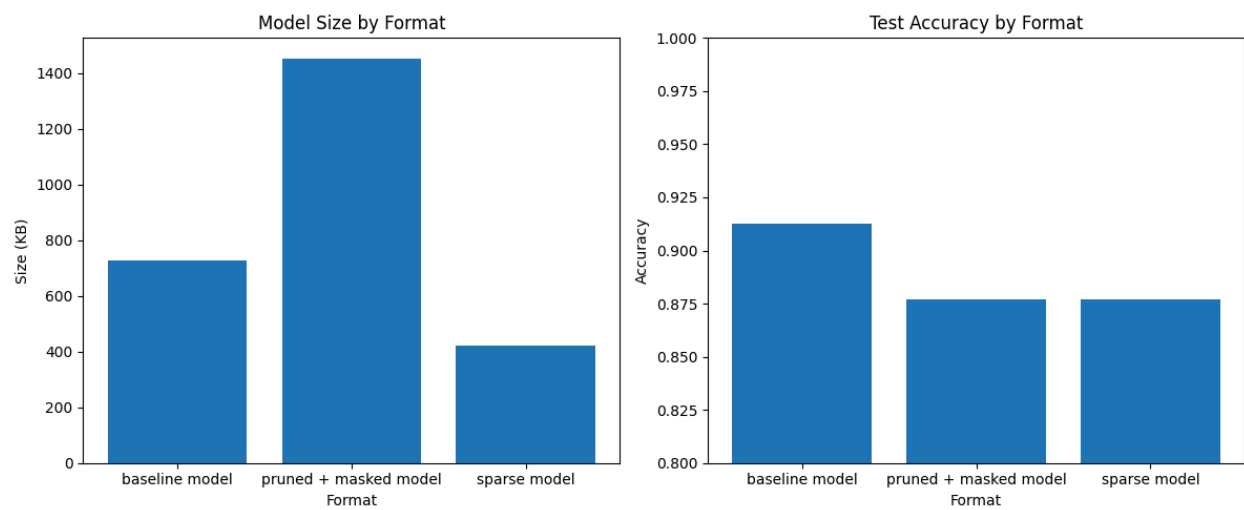
Probably the QAT Int 8 model. Despite being 1KB larger than its PTQ cousin, it gains an entire 1% accuracy bonus. I'm guessing the 1KB won't make or break anything,, but if it does, the 1% accuracy loss isn't a huge problem

Pruning

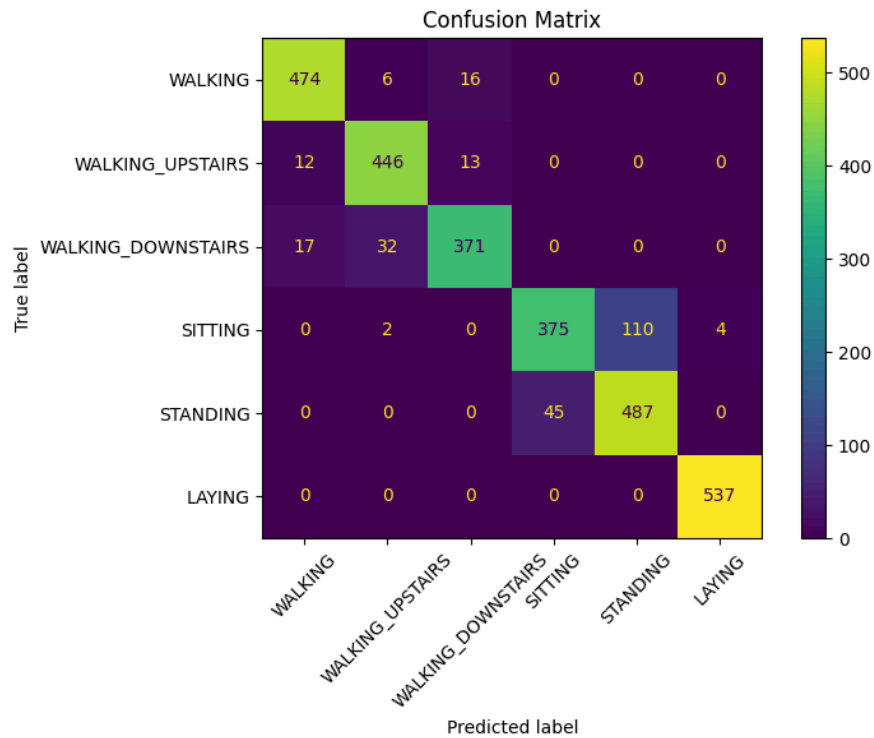
14. Submission Requirements

Submit the following:

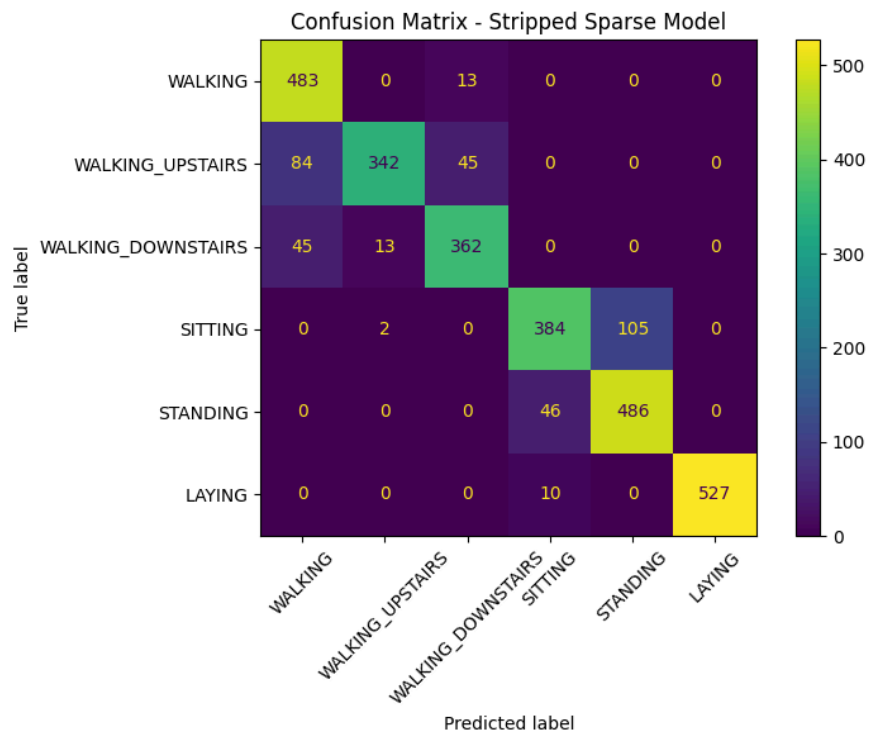
- your completed notebook,
- the generated `.tflite` files,
- output cells or screenshots showing
 - the comparison tables,



- confusion matrices for the baseline model



- and your final highlighted compressed model,



- and short written observations answering the summary questions.

Make sure your notebook runs from top to bottom without errors using the **Python (tinym1-arduino)** kernel.

13. Summary Questions

Write short answers to the following:

1. Did pruning alone reduce the TensorFlow Lite file size when the pruning wrappers were still attached?

No, the masks/wrappers take up a lot of additional storage space. It actually increases the file size

2. Why does `strip_pruning(...)` matter before export?

It removes the pruning wrappers. If you forgot this step your model would be super bloated and may even be too large for the device it's running on

3. Which model had the smallest file size in this notebook?

	Model	Size(KB)	Accuracy
0	baseline model	726.652344	0.912793
1	pruned + masked model	1454.000000	0.876824
2	sparse model	421.601562	0.876824

	Model	Size(KB)	Accuracy
0	pruned + masked FP32	1454.000000	0.876824
1	sparse FP32	421.601562	0.876824
2	pruned + masked float16	732.375000	0.877163
3	sparse float16	277.769531	0.877163

The float16 sparse model was the smallest.

4. Did float16 quantization noticeably change the test accuracy?

No, its a tenth of a percent difference in accuracy. ...And it somehow got more accurate anyway?

5. If you were deploying this model on a resource-constrained device, which version would you choose and why?

The sparse float16 is by far the best pick. You decrease your original model to less than half of its starting size, and only lose around 4 percentage points of accuracy. A more than reasonable trade to get it to run on a tiny device

You would also never have any reason to pick Sparse FP32 because it is around 150% the size of the float16 version but is nearly identical in performance

Distilling

21. Submission Requirements

Submit the following:

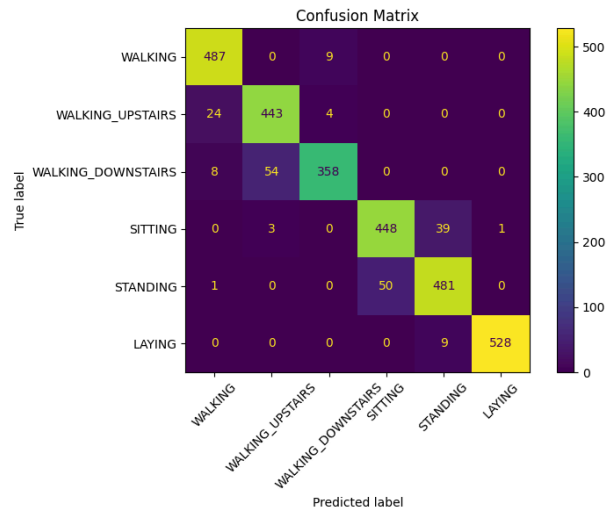
1. Your completed notebook
2. Screenshots of the most important results:
 - teacher accuracy

Test Accuracy: 0.9335

	precision	recall	f1-score	support
WALKING	0.92	0.98	0.95	496
WALKING_UPSTAIRS	0.95	0.93	0.94	471
WALKING_DOWNSTAIRS	0.97	0.93	0.95	420
SITTING	0.89	0.91	0.90	491
STANDING	0.88	0.90	0.89	532
LAYING	1.00	0.95	0.98	537
accuracy			0.93	2947
macro avg	0.94	0.93	0.93	2947
weighted avg	0.93	0.93	0.93	2947

- baseline student accuracy

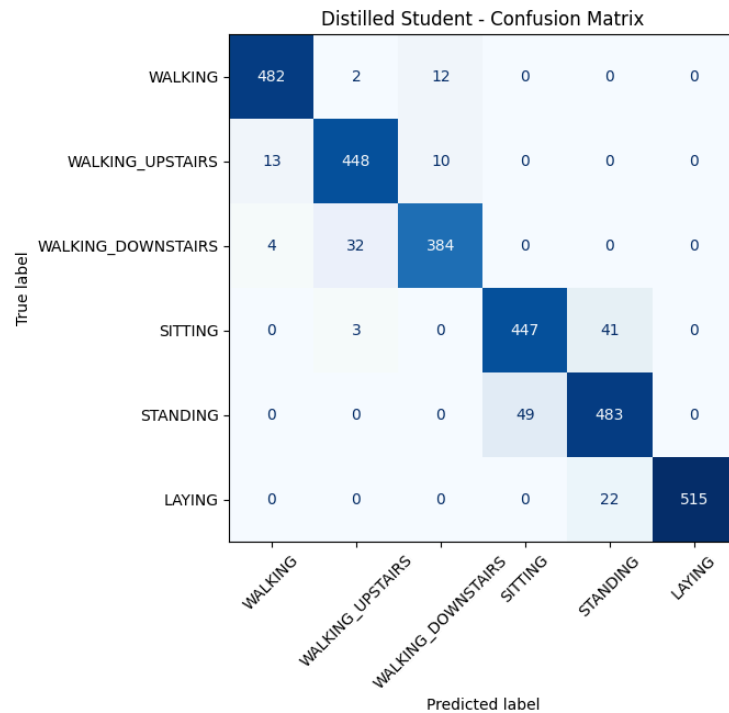
Epoch				
93/93 [=====] - 0s 3ms/step				
Test Accuracy: 0.9315				
	precision	recall	f1-score	support
WALKING	0.94	0.98	0.96	496
WALKING_UPSTAIRS	0.89	0.94	0.91	471
WALKING_DOWNSTAIRS	0.96	0.85	0.91	420
SITTING	0.90	0.91	0.91	491
STANDING	0.91	0.90	0.91	532
LAYING	1.00	0.98	0.99	537
accuracy			0.93	2947
macro avg	0.93	0.93	0.93	2947
weighted avg	0.93	0.93	0.93	2947



- distilled student accuracy

Distilled Student Test Accuracy: 0.9362

	precision	recall	f1-score	support
WALKING	0.9659	0.9718	0.9688	496
WALKING_UPSTAIRS	0.9237	0.9512	0.9372	471
WALKING_DOWNSTAIRS	0.9458	0.9143	0.9298	420
SITTING	0.9012	0.9104	0.9058	491
STANDING	0.8846	0.9079	0.8961	532
LAYING	1.0000	0.9590	0.9791	537
accuracy			0.9362	2947
macro avg	0.9369	0.9358	0.9361	2947
weighted avg	0.9371	0.9362	0.9365	2947



- final sparse INT8 model accuracy and size

	Model	Format	Test Accuracy	Model Size (KB)
0	Distilled Student TFLite	FP32	0.936206	149.210938
1	Pruned Distilled TFLite (with mask)	FP32	0.937224	299.730469
2	Stripped Sparse Distilled TFLite	FP32 + Sparse	0.937224	104.449219
3	Stripped Sparse Distilled TFLite	INT8 + Sparse	0.935867	54.125000

- The exported TensorFlow Lite model:
 - `distilled_stripped_sparse_int8.tflite`
- Short answers to the summary questions

20. Summary Questions

Answer the following in your lab report:

- How did the **baseline student** compare with the **distilled student**?

Honestly their performances were about the same. It has slightly higher accuracy overall, and some performance metrics are a bit different than the baseline model's, but surprisingly it isn't much to write home about

2. Did **knowledge distillation** help the smaller model retain performance?

Slightly but not as much as I'd expect

3. What happened to the model size after **pruning** and after **INT8 quantization**?

Massive decrease in model size, decreased to nearly a third of the original distilled model size

4. Which model would you choose for **Arduino deployment**, and why?

Technically depends on the device I am running it on, but most likely Sparse Int 8. If I really need that slight increase in accuracy and I have the space to run a larger model I'll use the Sparse FP32.

5. Why is the final **sparse INT8 model** a good TinyML deployment candidate?

Sparse Int 8 has the best of all worlds, a super tiny model size (1/3rd the original!) with only a negligible (0.02%!) decrease in accuracy

This is a dream come true for TML enjoyers. An absolutely tiny model who's small size has no negative impact on performance